



Öğrenci Bilgi Sistemi (SIS) - Kapsamlı Proje ve Mimari Dokümantasyonu

Bu belge, Öğrenci Bilgi Sistemi (Student Information System - SIS) projesinin temel mimari yapısını, veritabanı - nesne yönelimli programlama (OOP) entegrasyon kararlarını ve projenin genel işleyişini detaylandırmak üzere hazırlanmış kapsamlı bir mühendislik raporudur.

1. Giriş ve Projenin Amacı

Bu projenin temel amacı, bir üniversitenin akademik ve idari süreçlerini yönetecek olan yazılımın çekirdek (core) yapısını tasarlamaktır. Geliştirilen bu sistem, gerçek dünyadaki karmaşık veri ilişkilerini (Fakülte -> Bölüm -> Program -> Öğrenci vb.) yazılım ortamında güvenilir bir şekilde simüle etmektedir.

Tasarım sürecinde **SOLID** prensiplerinin temellerinden olan "Tek Sorumluluk Prensibi (Single Responsibility Principle)" gözetilmiş olup, her bir veri modeli kendi sınıfı (class) içerisinde bağımsız bir şekilde izole edilmiştir.

2. Nesne Yönelimli Programlama (OOP) Prensiplerinin Uygulanışı

Proje, baştan sona katı bir OOP mimarisi ile tasarlanmıştır. Bu mimarinin yapı taşları şunlardır:

2.1. Kapsülleme (Encapsulation)

Kapsülleme, nesnenin durumunu (state) korumak için kullanılan en temel güvenlik önlemidir. Tüm varlık sınıflarındaki veri alanları `private` erişim belirleyicisi (access modifier) ile dış dünyaya kapatılmıştır. Örneğin, bir öğrencinin not ortalamasına veya TC Kimlik numarasına dışarıdan doğrudan müdahale edilmesi (örnek: `student.nationalId = "123"`) engellenmiştir. Bunun yerine veriye erişim ve modifikasyon, kontrollü bir yapı olan **Getter ve Setter** metodları üzerinden sağlanmaktadır.

2.2. Yapıcı Metodlar (Constructors) ile Veri Bütünlüğü

Java'da nesne oluşturma aşamasında, nesnenin hafızada geçerli bir durumda olmasını garanti altına almak gereklidir. Bu projede, default (boş) constructor yerine, nesnenin ihtiyaç duyduğu tüm parametreleri zorunlu kılan **Parametrelili Constructor** yapısı kullanılmıştır. Böylece eksik bilgiyle (örneğin id'si olmayan bir öğrenci) nesne yaratılmasının önüne geçilmiştir.

2.3. Polimorfizm ve Object Sınıfı İlişkisi

Java'daki her sınıf gizli olarak `Object` sınıfından miras (inherit) alır. Bu projede her varlık sınıfı için `toString()` metodu `Override` (ezme) işlemi uygulanarak özelleştirilmiştir. Bu sayede bellek adresleri yerine konsolda okunabilir, formatlı veri dökümleri (loglar) elde edilmesi hedeflenmiştir.

3. Veritabanı İlişkilerinin Koda Yansıtılması (Relational Mapping)

Proje, sadece basit sınıflardan oluşmamakta; aynı zamanda bir İlişkisel Veritabanı (RDBMS) şemasını hafızada (in-memory) canlandırmaktadır.

3.1. UUID (Universally Unique Identifier) Yaklaşımı

Geleneksel artan tamsayı (Auto-increment integer) ID'ler yerine `java.util.UUID` sınıfı kullanılmıştır.

- **Neden UUID?** Dağıtık sistemlerde (distributed systems) ve asenkron veri kayıtlarında çakışmaları (collision) sıfıra indirmek için UUID modern bir standarttır.
- **Foreign Key (Yabancı Anahtar) Yönetimi:** Öğrencinin bağlı olduğu program, doğrudan bir Program nesnesi yerine UUID programId olarak tutulmuştur. Bu durum, veritabanı şemasındaki 1:N (Bire-Çok) ve M:N (Çoka-Çok) ilişkilerin tam bir yansımasıdır.

3.2. Sınıflar (Tablolar) Arası Hiyerarşi

Sistem tasarımı şu şekilde kurgulanmıştır:

- **Faculty (Fakülte):** Sistemin en üst birimi.
- **Department (Bölüm):** Fakülteye bağlı alt idari yapı. (facultyId ile bağlıdır).
- **Program (Öğretim Programı):** Bölüme bağlı eğitim müfredatı. (departmentId ile bağlıdır).
- **Student (Öğrenci) & Instructor (Öğretim Üyesi):** Program ve bölümlerle eşleşen temel insan kaynakları.

4. Tip Güvenliği İçin Enum Mimarisi

Kullanıcının veya sistemin metin tabanlı (String) girişler yapması, veri tabanında tutarsızlıklara ("Güz", "GÜZ", "guz" karmaşası gibi) yol açar. Bu sorunu çözmek için projede kapsamlı bir **Enum** mimarisi kurulmuştur.

Tanımlanan Enum Sınıfları ve Görevleri:

- **Gender (E, K):** Cinsiyet verisinin standardizasyonu.
- **StudentStatus (AKTIF, MEZUN, ASKI, AYRILDI):** Öğrencinin akademik döngüdeki durum takibi.
- **DegreeLevel (ONLISANS, LISANS...):** Programların seviyelerinin belirlenmesi.
- **Semester (GUZ, BAHAR, YAZ):** Akademik dönemin hatasız işlenmesi.
- **CourseType (ZORUNLU, SECMELI, ASD):** Ders seçim süreçlerindeki mantıksal ayrımlar.
- **PrerequisiteType (ZORUNLU, ONERILEN):** Ders önkoşul hiyerarşisinin belirlenmesi.
- **InstructorTitle (DR, DOC_DR...):** Akademik unvanların kategorizasyonu.

Enum kullanımı, derleme zamanı (compile-time) kontrolü sağlayarak String kullanımından kaynaklanabilecek potansiyel hataları (Run-time Exceptions) tamamen ortadan kaldırmıştır.

5. Main.java: Sistem Simülasyonu ve Etkileşim

Main.java sınıfı, tasarlanan bu mimarinin test edildiği ve simüle edildiği çalıştırıcı (driver) sınıftır.

5.1. Dinamik Veri Yapıları: Neden ArrayList?

Öğrenci kapasitesi önceden bilinemeyeceği için, boyutu sabit olan diziler (Array []) yerine, java.util.List arayüzünü (interface) uygulayan java.util.ArrayList sınıfı kullanılmıştır. Bu yapı, O(1) zaman karmaşıklığı ile hızlı okuma ve dinamik büyüme/küçülme avantajı sağlar.

5.2. CLI (Komut Satırı Arayüzü) ve Akış Kontrolü

Sistemde kullanıcı etkileşimi Scanner sınıfı ile asenkron olmayan bir bloklayıcı (blocking) I/O yöntemiyle sağlanmaktadır.

- Menü sonsuz bir while döngüsü içerisine alınarak sistemin sürekli (persistent) bir uygulama (application) gibi davranması sağlanmıştır.

- Durum geçişleri için `switch-case` kullanılarak kodun döngüsel karmaşıklığı (cyclomatic complexity) düşük tutulmuş, okunabilirlik maksimuma çıkarılmıştır.
-

6. Java API Kullanımları ve Tarih Formatları

Projede güncel Java (Java 8 ve üzeri) kütüphaneleri aktif olarak kullanılmıştır:

- `java.time.LocalDate` : Sadece tarih (Yıl-Ay-Gün) içeren doğum tarihi (`birthDate`) veya işe giriş tarihi (`hireDate`) gibi alanlar için kullanılmıştır. Eski `java.util.Date` sınıfının aksine değiştirilemez (immutable) olduğu için daha güvenlidir.
 - `java.time.LocalDateTime` : Saat bilgisinin de kritik olduğu kayıt oluşturma tarihleri (`createdAt`) için tercih edilmiştir.
-

7. Gelecek Geliştirmelere (Extensibility) Yönelik Değerlendirme

Şu anki mimari "Gevşek Bağlılık" (Loose Coupling) ve "Yüksek Uyum" (High Cohesion) standartlarına göre dizayn edildiğinden, projeye ilerleyen süreçlerde eklenecek olan:

- **JDBC / Hibernate Veritabanı Entegrasyonları,**
- **JavaFX veya Spring Boot ile Arayüz (GUI / Web API) Geliştirmeleri,**
- **Genişletilmiş Hata Yönetimi (Exception Handling)**

gibi modüller mevcut çekirdek kodu (Core Classes) bozmadan kolayca entegre edilebilecek yapıdadır. Proje, endüstri standartlarında bir yazılımın prototipi olarak başarıyla tasarlanmıştır.